

Métropole – 2025 – sujet2 - Correction

Exercice 1 (6 points)

Partie A – Lecture et décodage d'un arbre de codage

1. Le code binaire associé au symbole espace () est obtenu en suivant le chemin de la racine à la feuille correspondante dans l'arbre. On lit ainsi le mot binaire 111.

Commentaire : on attend ici que l'élève sache lire un code binaire directement sur un arbre de codage, sans effectuer de calcul.

2. Pour décoder la chaîne binaire 000111010111110011001, on parcourt l'arbre depuis la racine jusqu'à atteindre une feuille, puis on recommence à la racine. Le message obtenu est : « je suis ».

Commentaire : la méthode de décodage est essentielle. Une erreur fréquente consiste à découper arbitrairement la chaîne binaire.

3. Le parcours adapté pour classer les symboles par longueur de code croissante est le parcours en largeur.

Commentaire : le parcours en largeur visite les nœuds niveau par niveau, ce qui correspond à un classement par profondeur.

Partie B – Codage de Shannon-Fano

4. La somme totale des occurrences vaut 22. La séparation est effectuée de façon à obtenir deux groupes de sommes aussi proches que possible : 11 et 11.

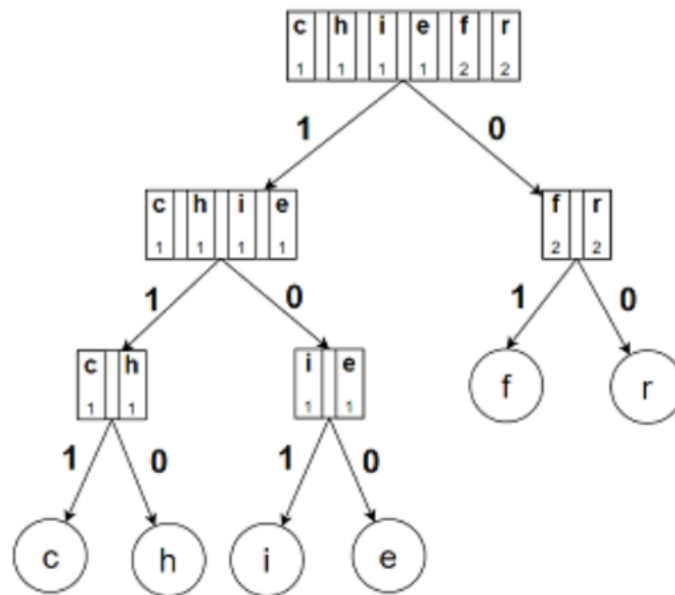
Commentaire : le principe du codage de Shannon-Fano repose sur l'équilibrage des probabilités.

5. La hauteur de l'arbre est égale à 5. Elle correspond à la longueur maximale d'un mot binaire.

Commentaire : la hauteur donne directement la longueur du code le plus long.

6. Le codage ASCII nécessite $22 \times 8 = 176$ bits. Le codage de Shannon-Fano utilise environ deux fois moins de bits, car les symboles fréquents sont codés avec moins de bits.

7. Dessin de l'arbre de codage



Partie C – Programmation Python

8. Cette fonction construit un dictionnaire associant à chaque symbole son nombre d'occurrences.

```
def creer_dico_occ ( texte ) :  
    dico = {}  
    for symbole in texte :  
        if symbole in dico :  
            dico[symbole] = dico[symbole] + 1  
        else :  
            dico[symbole] = 1  
    return dico
```

Commentaire : on parcourt le texte caractère par caractère et on met à jour le dictionnaire.

9. Fonction somme_occ

```
def somme_occ ( tab ) :  
    s = 0  
    for _, nb in tab :  
        s = s + nb  
    return s
```

Commentaire : cette fonction calcule la somme des occurrences d'un tableau de couples.

10. Fonction Shannon

```
def shannon ( symbole, tab ) :  
    if len ( tab ) == 1 :  
        return ""  
    t1, t2 = separe ( tab )  
    if symbole in [x[0] for x in t1] :  
        return "1" + shannon ( symbole, t1 )  
    else :  
        return "0" + shannon ( symbole, t2 )
```

Commentaire : la terminaison est assurée car la taille du tableau diminue à chaque appel récursif.

11. Terminaison de la fonction récursive shannon

La terminaison de la fonction récursive shannon est garantie par le fait que, lors de chaque appel récursif, le tableau tab est remplacé par l'un des deux sous-tableaux retournés par la fonction separe(tab).

Ces sous-tableaux sont strictement plus petits que le tableau initial. La récursion s'arrête lorsque tab ne contient plus qu'un seul symbole, ce qui correspond au cas de base len(tab) == 1.

Commentaire : l'élève doit identifier à la fois le cas de base et la diminution d'une grandeur (ici la taille du tableau) à chaque appel récursif.

12. Fonction encode_shannon

La fonction encode_shannon construit le mot binaire associé à un texte en utilisant successivement les fonctions fournies.

```
def encode_shannon ( texte ) :  
    dico = creer_dico_occ ( texte )  
    tab = creer_tab_trie ( dico )  
    code = ""  
    for symbole in texte :  
        code = code + shannon ( symbole, tab )  
    return code
```

Commentaire : le dictionnaire d'occurrences est calculé une seule fois. Pour chaque symbole du texte, on concatène son code binaire obtenu grâce à la fonction shannon.